数字系统设计实验指导书

廖荣　　编

华南理工大学教材供应中心

2016年9月

目　　录

实验一、VHDL 程序结构的熟悉 ……3

实验二、基本电路模块设计（组合、时序） ……27

实验三、基于状态机的交通灯控制 ……39

实验四、按键控制的状态机设计 ……45

FPGA 黑金开发平台 AX301 用户手册 ……47

AX301 使用指南 ……80

安装 MODELSIM 及仿真 ……93

LED 流水灯实验 ……114-154

实验一　VHDL 程序结构的熟悉

一 实验目的：

1. 熟悉 Quartus 软件的使用

2. 熟悉 VHDL 程序结构

二 实验内容：

按新教材实例（P120 页开始，详见下面的附件资料）操作，掌握建立工程、设计输入、编译、时序仿真等开发流程；至有仿真结果为止。要理解 VHDL 代码意思，将软件和硬件结合起来，掌握开发流程。特别提醒：工程名一定要同顶层文件名（总电路图）一致；实体名一定要同当前待编译文件名一致；当前文件做编译之前一定要先设为顶层文件（点击 Project 中 Set as Top-Level Entity）。

附新教材实例资料：SCUT-EDA 开发平台

为了顺利开展日常教学工作与满足工程实践的需要，华南理工大学自主开发了 EDA 实验平台 younever_v1.2，如图 1 所示。该平台选用了 Altera 公司的 Cyclone ii 芯片，配套了丰富的硬件资源，主要包括电源稳压电路、8 位七段数 码管、1602 液晶屏接口、音频接口、串行配置芯片 EPCS16、温度传感器、VGA 接口、Ps2 接口、9 针串口、EEPROM、红外接收与发送电路、下载接口、DM9000A 驱动的网卡接口等，其顶层 PCB 如图 2 所示。该实验平台配置灵活，各模块电 路独立工作，可通过跳线设置是否与芯片连接，能够完成多种实验与课程设计。

在实验的过程中，读者可以根据条件选择适合的硬件平台。书中工程实例部分在进行管脚配置的时候，是根据上述平台设定的，对于不同的实验平台，读者可以根据实际情况，作出相应的修改。

6.1 Quartus II 软件设计流程

6.2.1 基于 Quartus II 的数字系统设计流程

Altera Quartus II 设计软件提供完整的多平台设计环境，能够直接满足特定设计需要，为可编程芯片系统(SOPC)提供全面的设计环境。Quartus II 软件含有 FPGA 和 CPLD 设计所有阶段的解决方案。Quartus II 的开发流程如图3 所示。

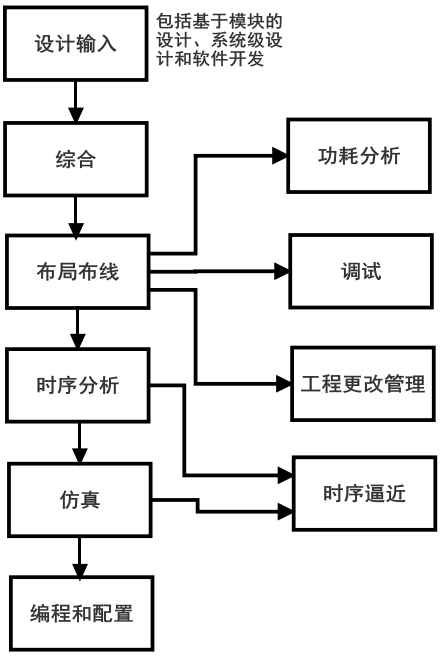


图3 QuartusⅡ的开发流程

图3 中的流程：设计输入->综合->布 线布局->时序分析->仿真->编程配置，在 Quartus II 中综合、布线布局、时序 分析都包含在编译中，也就是在点击 Start Complication 后，软件会自动完成 这三部分的功能。设计输入一般有文本输入、图形输入、网表输入等几种方法， 仿真之前需要编辑好输入信号激励文件，编程配置则需要根据开发板的说明书来 对系统的输入输出管脚进行配置。

此外，Quartus II 软件为设计流程的每个阶段提供 Quartus II 图形用户界面、EDA 工具界面以及命令行界面。可以在整个流程中只使用这些界面中的一个，也可以在设计流程的不同阶段使用不同界面。

6.2.2 Quartus II 软件使用介绍

Quartus II 软件可以完成设计流程的所有阶段。它是一个全面的、易于使用的独立解决方案。图4 显示 Quartus II 图形用户界面为设计流程每个阶段所提供的功能。

设计输入
* 文本编辑器
* 模块和符号编辑器
* Mega Wizard plug-inManager

约束输入
* 分配编辑器
* 引脚规划器
* Settings对话框
* 平面布局图编辑器
* 设计分区窗口

综合
* 分析与综合
* VHDL, Verilog HDL&AHDL
* 设计助手
* RTL查看器
* 技术映射查看器
* 渐进式综合

布局布线
* 适配器
* 分配编辑器
* 平面布局图编辑器
* 渐进式编译
* 报告窗口
* 资源优化顾问
* 设计空间管理器
* 芯片编辑器

时序分析
* 时序分析仪
* 报告窗口
* 技术映射查看器

仿真
* 仿真器
* 波形编辑器

编程
* 汇编程序
* 编程器
* 转换程序文件

系统级设计
* SOPC Builder
* DSP Builder

软件开发
* Software Builder

基于模块的设计
* LogicLock窗口
* 平面布局图编辑器
* VQM Writer

EDA界面
* EDA Netlist Writer

功耗分析
* PowerPlay Power Analyzer工具
* PowerPlay Early Power Estimator

时序逼近
* 平面布局图编辑器
* LogicLock窗口
* 时序优化顾问
* 设计空间管理器
* 渐进式编译

调试
* Signal Tap II
* Signal Probe
* 在系统存储内容编辑器
* RTL查看器
* 技术映射查看器
* 芯片编辑器

工程更改管理
* 芯片编辑器
* 资源属性编辑器
* 更改管理器

图4 Quartus II 图形用户界面的功能

1. 建立工程

1)启动 Quartus II 软件后，就会出现如图5 所示的用户图形界面。

2)使用 New Project Wizard（File 菜单）或 quartus_map 可执行文件建立

新工程。建立

新工程时，指定工程工作目录，分配工程名称，指定顶层设计实体的名称。在用户图形界面点击 File 菜单，在 File 下拉菜单中点击 New Project Wizard，如图6所示。点击 New Project Wizard 后，出现如图7 所示的界面。

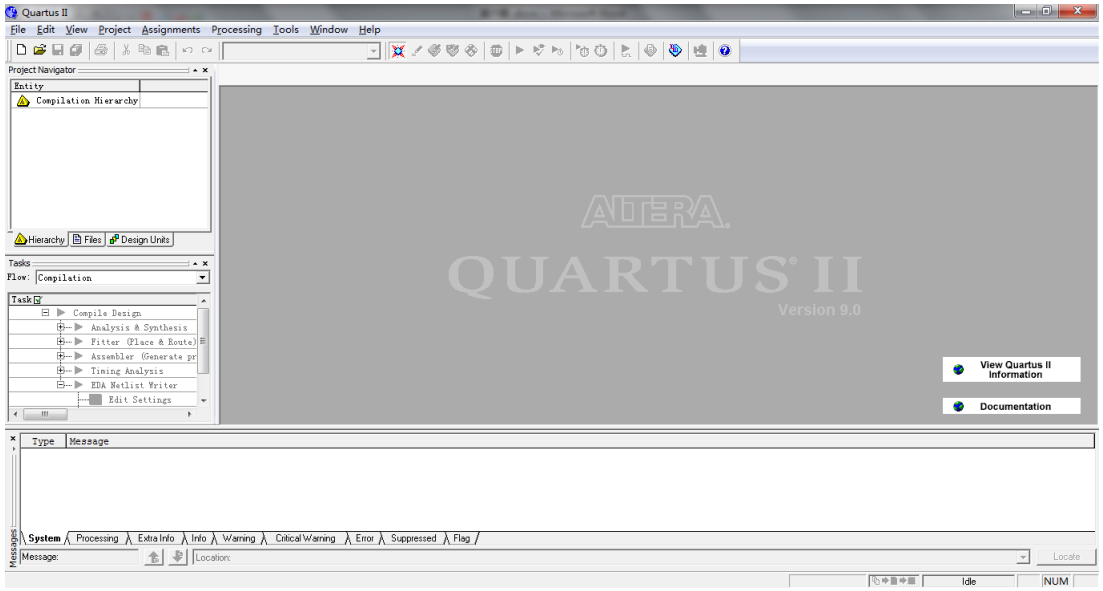


图5 用户图形界面

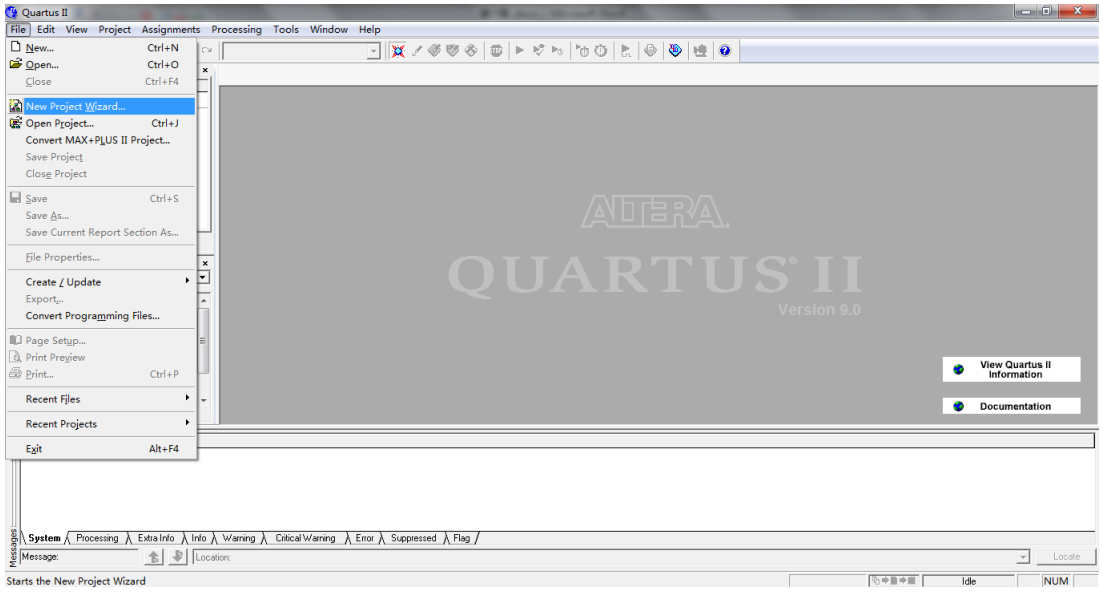


图6　点击 File 下拉菜单中的 New Project Wizard

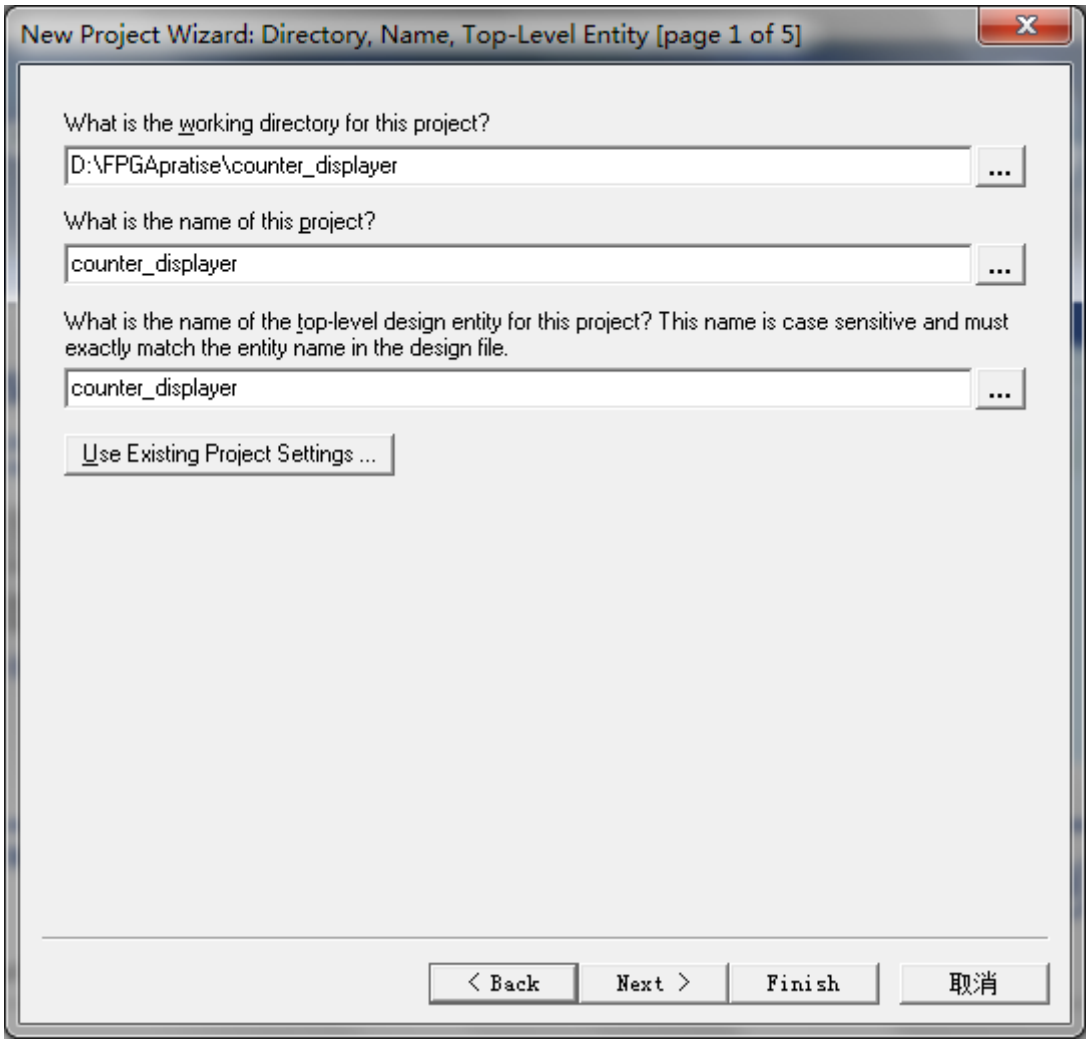


图7　新建工程向导：目录、名称和顶层设 计实体名

在指定路径之前，可以先新建一个工程文件夹，如在这里我们新建一个工程文件夹并命名为 counter_displayer，将工程取名为 counter_displayer，因为工程的顶层设计实体名应与工程名一致，所以工程的顶层设计实体名也是 counter_displayer，设置后如图7 所示，然后点击 next。

3)出现如图 8 所示的添加设计文件的对话框。

当然，在这里可以选择不添加文件而直接点击 next 进入下一步，因为在 Quartus II 用户图形界面中也可以通过点击 assignment-> setting-> files 来添加您所需要的文件。所以在这里

我们直接点击 next 进入下一步。

4)出现如图9 所示的器件系列设置对话框。

对于不需要下载到开发板而只想软件仿真的设计，在这里可以不用指定特定的器件，让软件自动选择适当的器件，但对于需下载到开发板来实现的设计，就应该在这里选择与开发板上对应的芯片。例如，若开发板上用的是 Cyclone II 系列的 EP2C8Q208C8 的芯片，那么就应该在下拉菜单 Family 下应选择 Cyclone II，在 Available devices 中选择 EP2C8Q208C8，然后点击 Next 进入下一步。

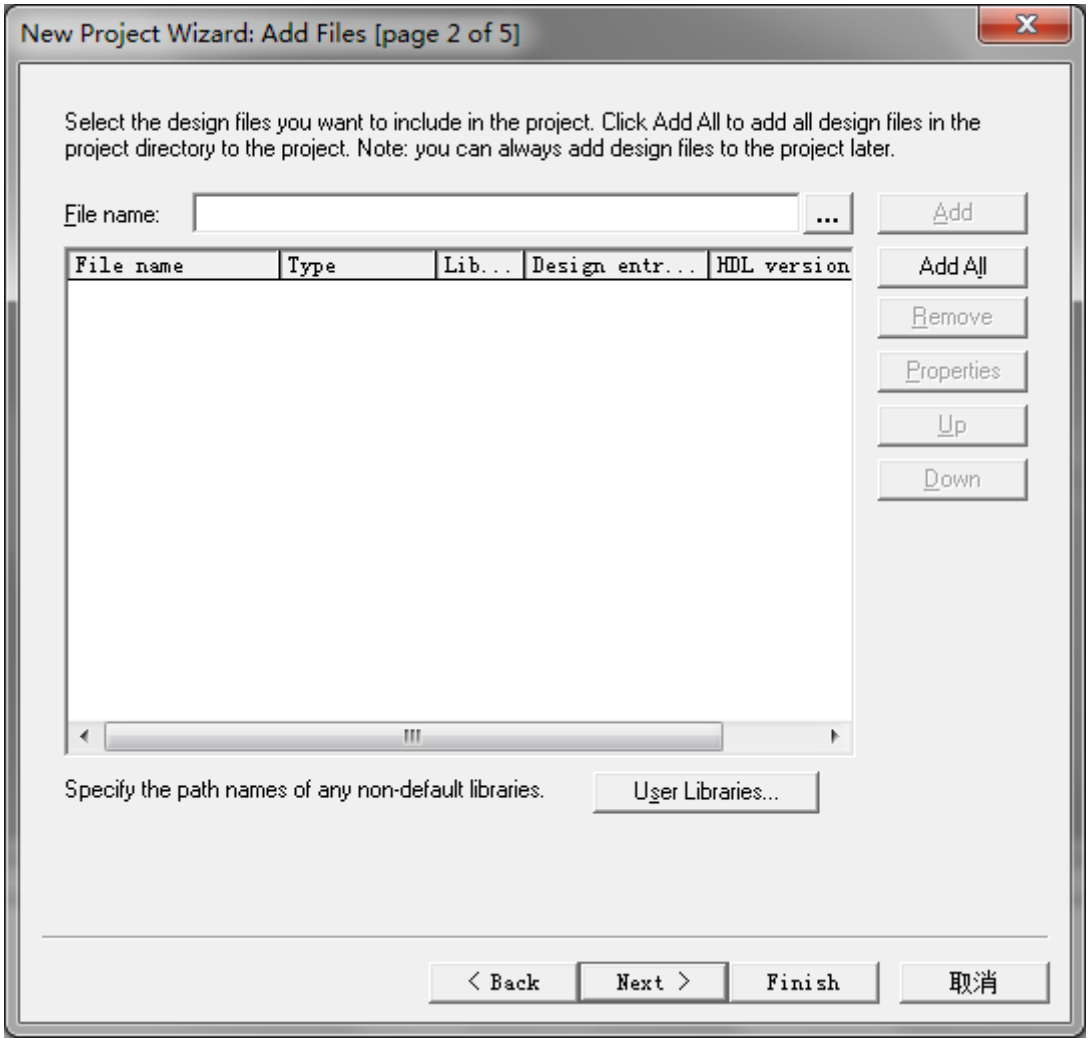


图8　新建工程向导：添加设计文件对话框

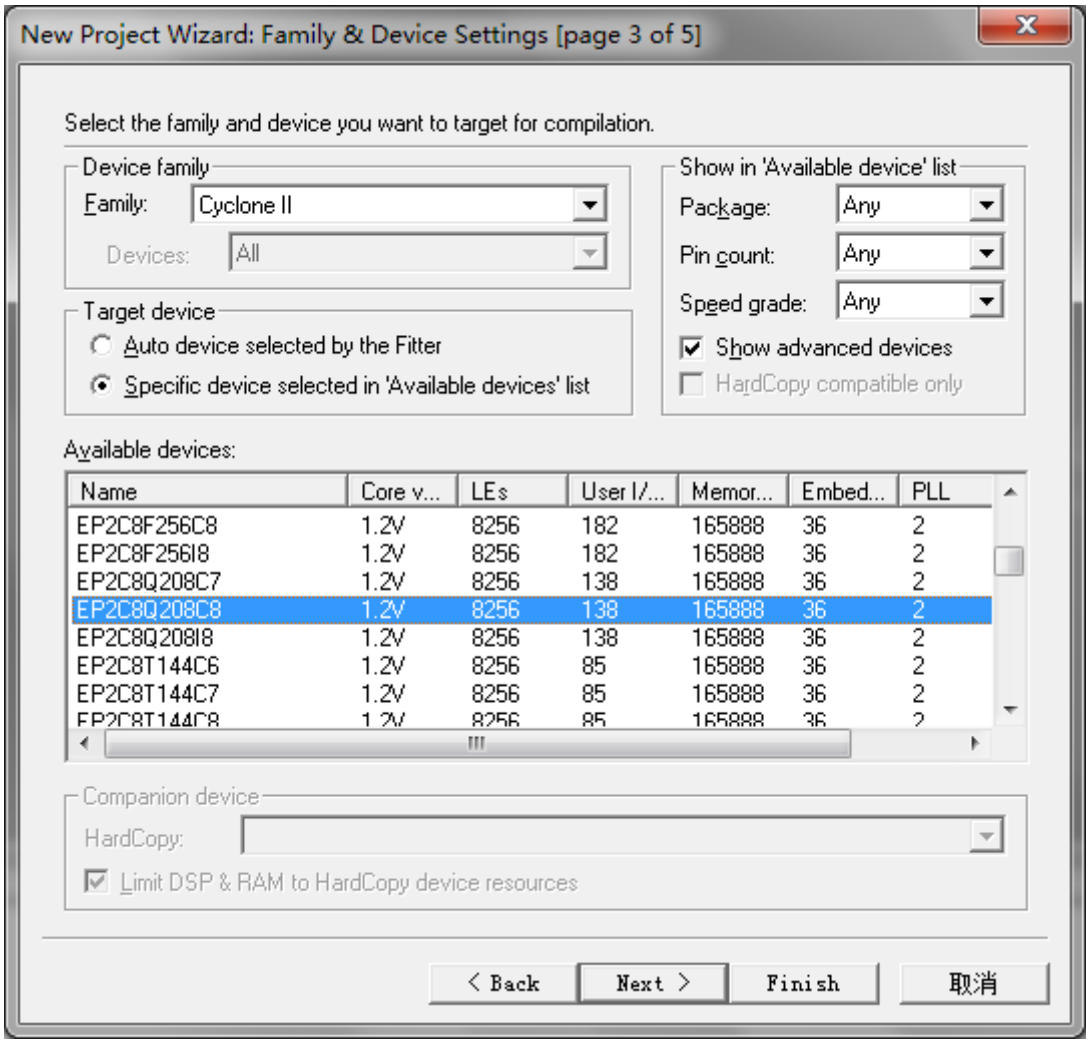


图9　新建工程向导：器件系列设置对话框

5)出现如图10 所示的 EDA 工具设置对话框。

Quartus II 软件本身包含了一套完整的开发流程，从设计输入、分析与综合、功能仿真、适配、时序分析、时序仿真，到下载都可以在 Quartus II 中完成，对于没有特别的要求可以不调用其他的 EDA 软件，在这里直接选择 Next 进入下一步。

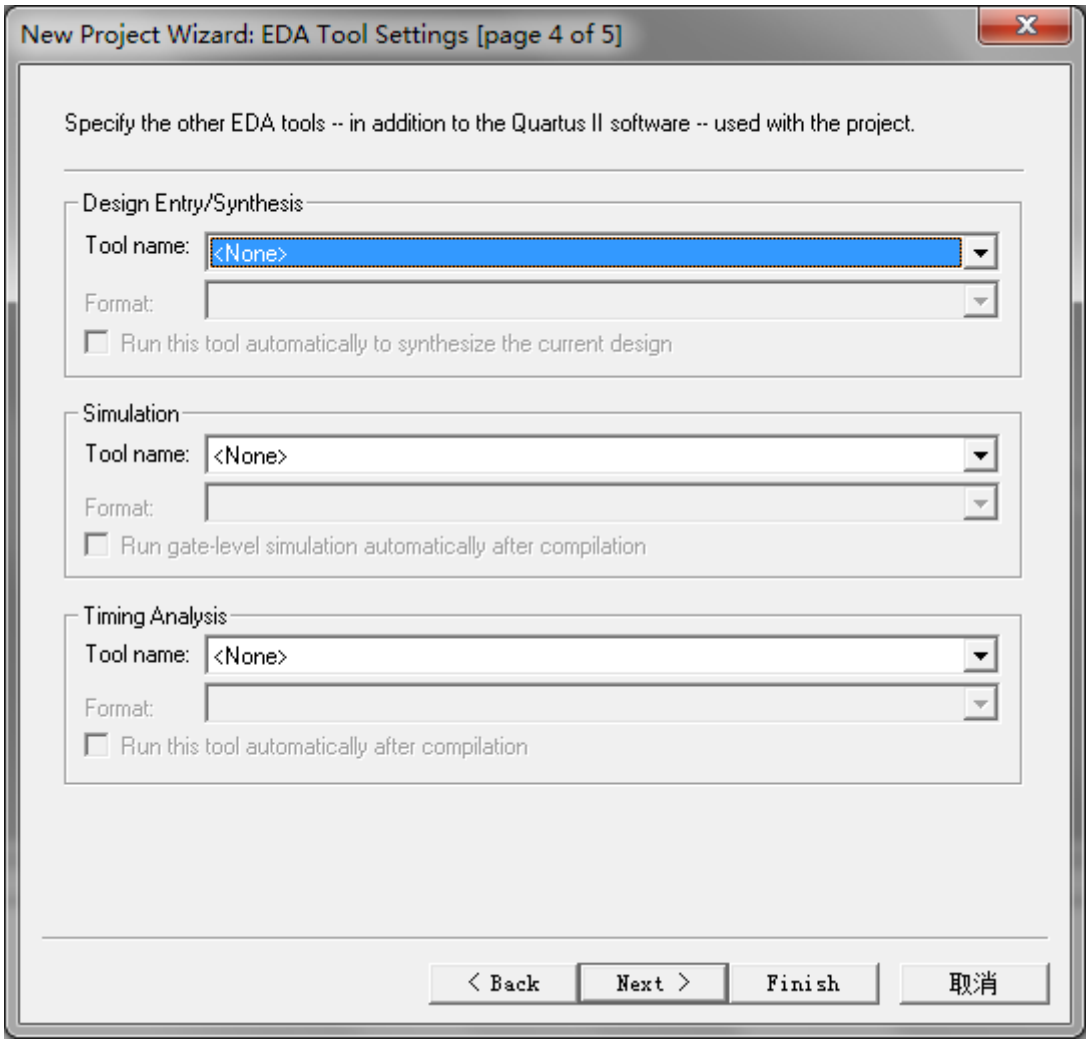


图10　新建工程向导：EDA 工具设置对话框

6) 出现如图11 所示的新建工程摘要界面。

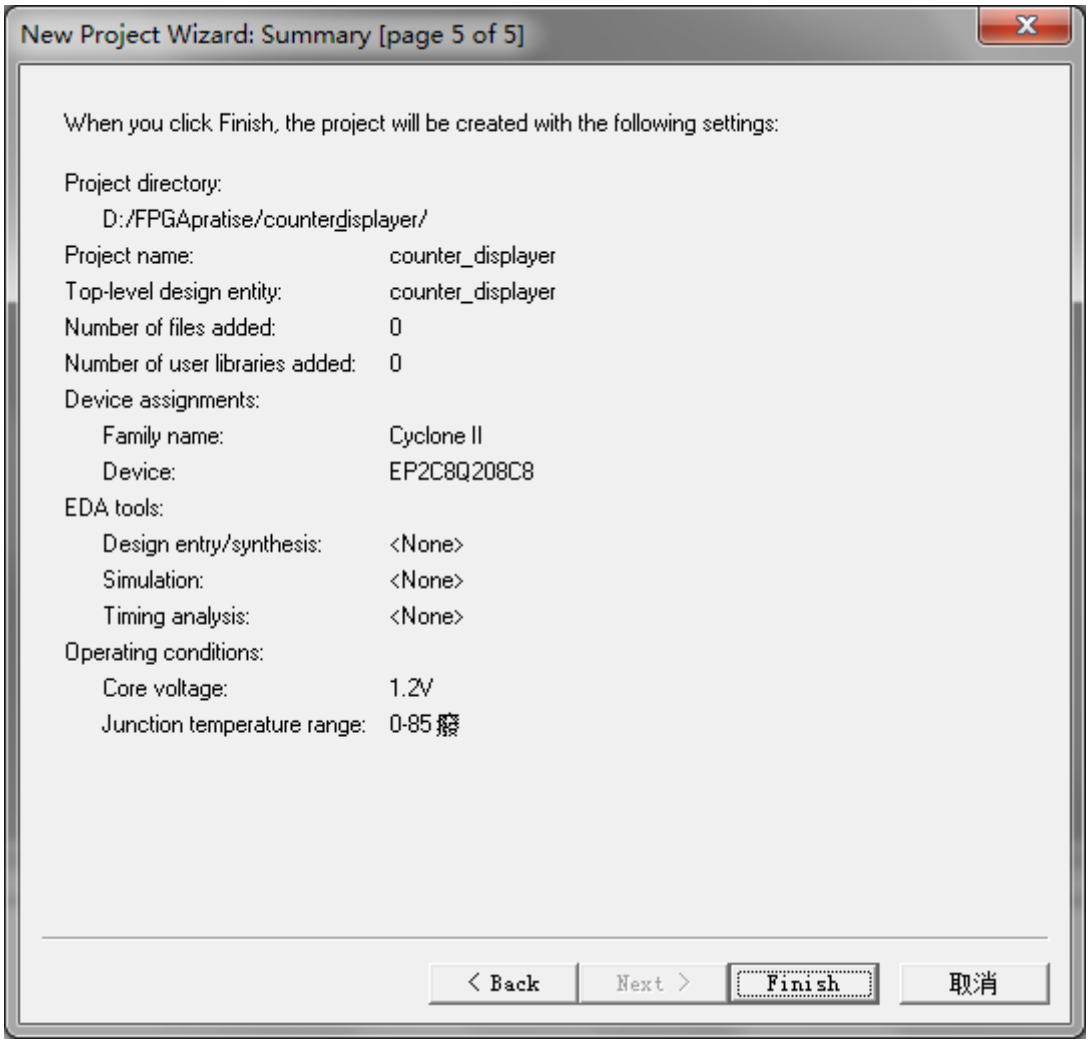


图11　新建工程的摘要

至此，一个新工程已经建立好，点击 Finish 可以进入到此工程的编辑环境。

这个设计可以分为两个部分：第一部分就是十六进制计数器；第二部分是七段数码管的显示部分。对于计数器就直接调用软件库里面的模块，显示部分实际上也就是个译码电路，所以接下来我们将详细说明这两部分的设计来介绍从设计输入到下载的整个流程。

2. 设计输入

可以使用 Quartus II 软件在 Quartus II Block Editor 中建立设计，或使用 Quartus II Text Editor 通过 AHDL、Verilog HDL 或 VHDL 设计语言来建立 HDL 设计。Quartus II 软件还支持采用 EDA 设计输入和综合工具生成的 EDIF InputFiles(.edf)或

Verilog Quartus Mapping Files (.vqm)建立的设计。另外，还可以在 EDA 设计输入工具中建立 Verilog HDL 或 VHDL设计，生成 EDIF 输入文件和 VQM 文件，或在 Quartus II 工程中直接使用 Verilog HDL 或VHDL 设计文件。

在这个设计中，采用 Quartus II Block Editor 和 Quartus II Text Editor 两种输入组合来完成设计。

(1)设计译码器电路首先设计译码器电路，采用文本输入方式来描述此电路，它的输入来自十六进制计数器的输出：qa、qb、qc 和 qd，译码器的输出连接到七段数玛管，这是个典型组合逻辑电路。在 Quartus II 图形用户界面下：点击 File->New 后，在如图12 所示的对话框中 选择 VHDL File。

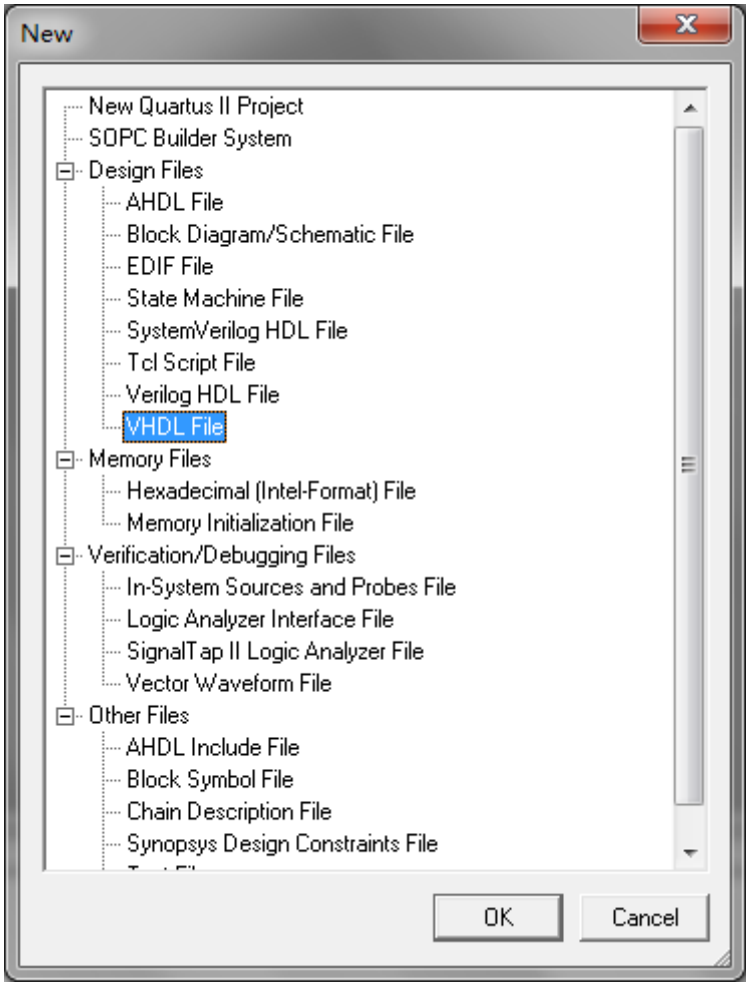


图12　新建文件类型选择

双击 VHDL File 之后，保存文件 VHDL1.vhd，文件名读者可以自己命名，但文件名一般都是和程序或电路相关的，如在这里我们取名为 decode，保存后出现如图13 所的代码编辑窗口。

图13 中，decode. vhd 是保存后的文 件名，而 counter displayer 是整个工程的工程名，因为在一个工程下可以包含 多个文件（.vhd，.bsf 等等），接下来就是编写此组合逻辑电路来实现译码的功 能，如图14 所示。

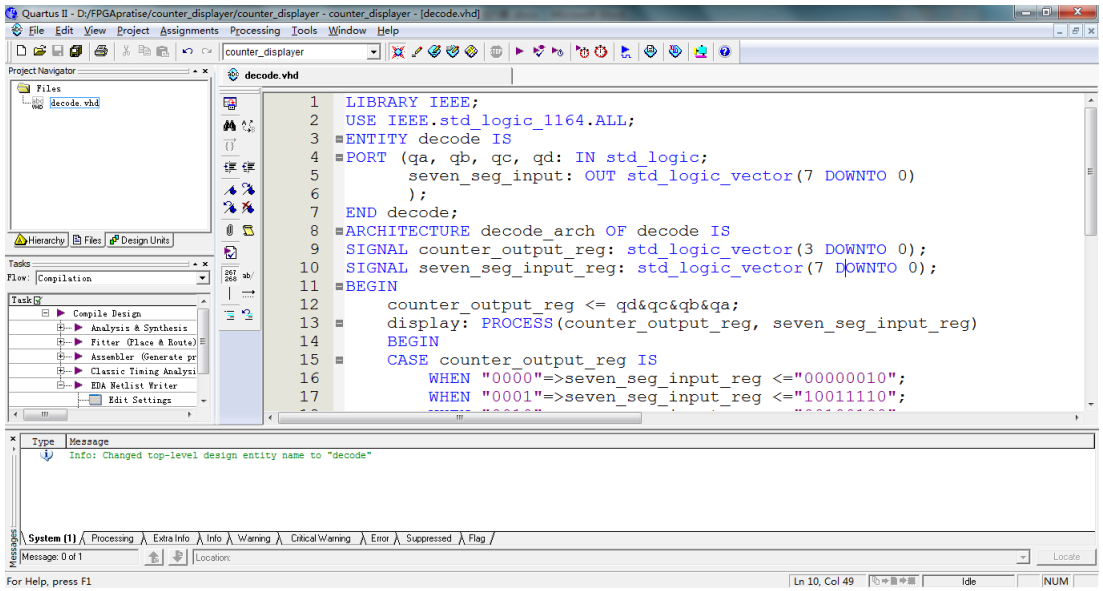


图13 代码编辑窗口

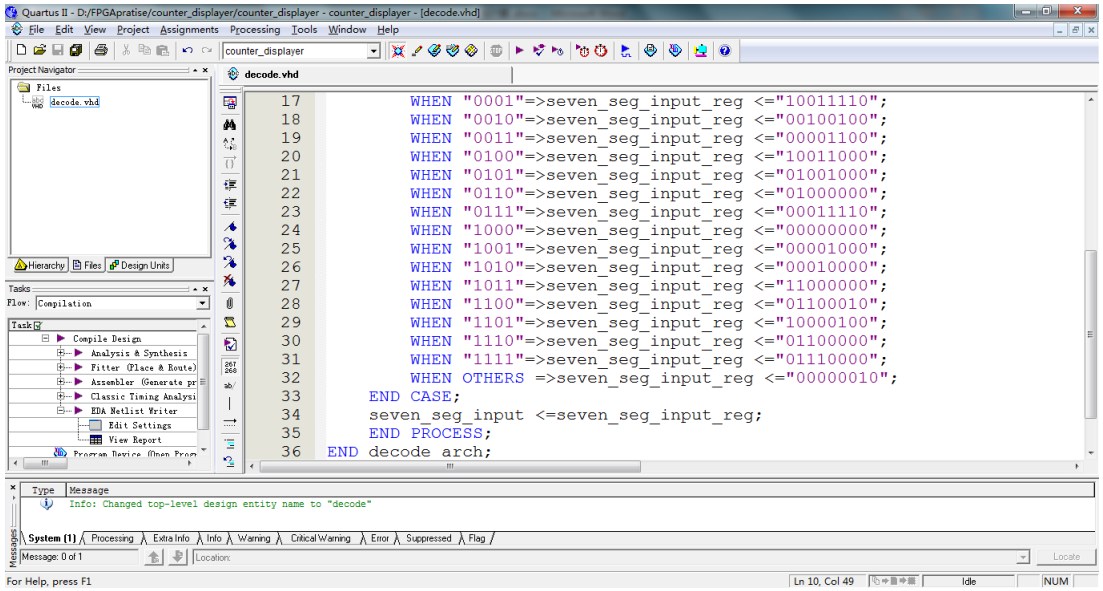


图14 编写程序完整程序如下：

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
ENTITY decode IS
PORT (qa, qb, qc, qd: IN std_logic;
      seven_seg_input: OUT std_logic_vector(7 DOWNTO 0)
      );
END decode;
ARCHITECTURE decode_arch OF decode IS
SIGNAL counter_output_reg: std_logic_vector(3 DOWNTO 0);
SIGNAL seven_seg_input_reg: std_logic_vector(7 DOWNTO 0);
BEGIN
    counter_output_reg <= qd&qc&qb&qa;
    display: PROCESS(counter_output_reg, seven_seg_input_reg)
    BEGIN
    CASE counter_output_reg IS
        WHEN "0000"=>seven_seg_input_reg <="00000010"; WHEN
        "0001"=>seven_seg_input_reg  <="10011110";  WHEN
        "0010"=>seven_seg_input_reg  <="00100100";  WHEN
        "0011"=>seven_seg_input_reg  <="00001100";  WHEN
        "0100"=>seven_seg_input_reg  <="10011000";  WHEN
        "0101"=>seven_seg_input_reg  <="01001000";  WHEN
        "0110"=>seven_seg_input_reg  <="01000000";  WHEN
        "0111"=>seven_seg_input_reg  <="00011110";  WHEN
        "1000"=>seven_seg_input_reg  <="00000000";  WHEN
        "1001"=>seven_seg_input_reg  <="00001000";  WHEN
        "1010"=>seven_seg_input_reg  <="00010000";  WHEN
        "1011"=>seven_seg_input_reg <="11000000";
```

```
        WHEN "1100"=>seven_seg_input_reg <="01100010";
        WHEN "1101"=>seven_seg_input_reg <="10000100";
        WHEN "1110"=>seven_seg_input_reg <="01100000";
        WHEN "1111"=>seven_seg_input_reg <="01110000";
        WHEN OTHERS =>seven_seg_input_reg <="00000010";
    END CASE;
    seven_seg_input <=seven_seg_input_reg;
    END PROCESS;
END decode_arch;
```

程序中信号的命名因人而异，但主要是为了方便自己阅读程序，当然对于复杂的程序，应养成给程序添加注释的习惯，以便日后查阅或供他人阅读。其中 qa、qb、qc、和 qd 来自计数器的输出；seven_seg_input 是译码器的输出信号，输入到七段数码管。

程序输入完之后，就可以对这段代码进行编译了。但应该注意的是，先把 decode.vhd 设为顶层文件再进行编译，不然编译报错说找不到顶层文件，设置方法：在 Quartus II 软件中点击 Project 菜单，出现如

图15 所示的对话框，并选择 Set as Top-Level Entity 或直接使用组合键 Ctrl+ Shift +J，然后就可以点击编译进 行编译了。

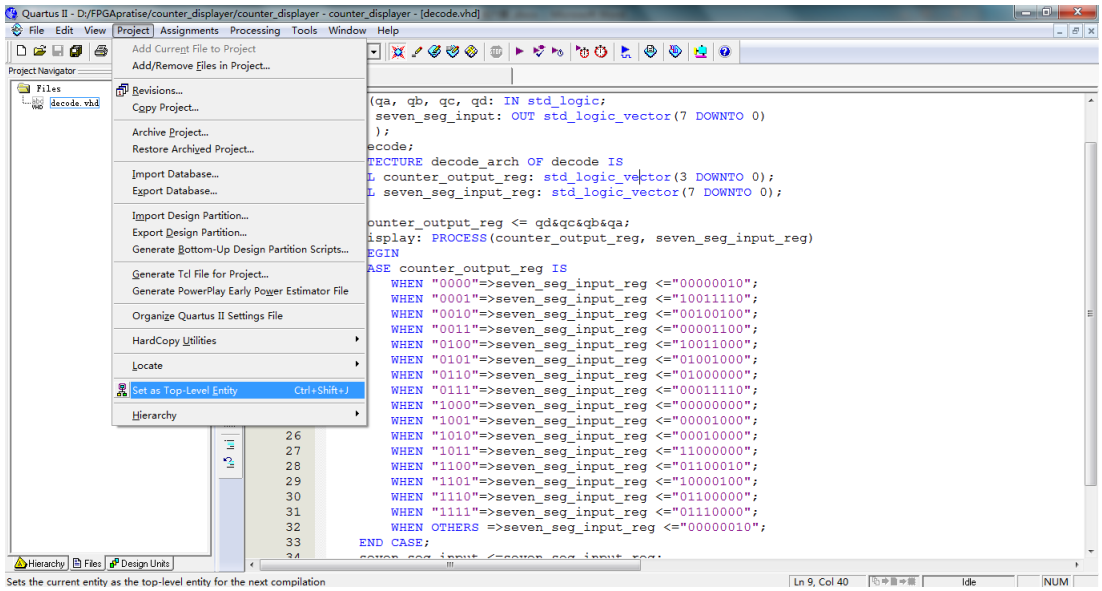


图15 设为顶层文件

编译完之后，将此程序生成 block 块，在设计整体功能时要调用到这个模块，生成 block 块的方法是：在用户界面中点击 decode.vhd 文件，使之处于当前活动窗口，然后点击 File-> Create\Update->Create Symbols for Current File，等待 symbol 的生成，如图16 所示。

成功生成之后会有对话框提示：Create symbol file was successful，生成的 block 会自动添加到 Project-> decode 下，如图17 所示。

到这一步，译码器电路就已经完成了。

(2)建立观察模块为了仿真的方便，我们生成另一个小模块，在仿真的时候便于我们观察计数器的计数过程。描述如下：把计数器的输出信号 qa、qb、qc 和 qd 作为译码器输入的同时，又将此 4 个信号作为这个小模块的输入，小模块的输出信号是 result，也就是计数器的输出，信号 result 的值是从 0-直到 15，其实和译码器的输出是一样的，不同的是：小模块 counter_output 的输出信号 result 是为了在时序仿真是便于观察计数器的计数，而译码器 decode 的输出信号 seven_segment[7..0]在下载配置的时候是和七段数码管相对应的，直接控制数码管上显示的计数值。

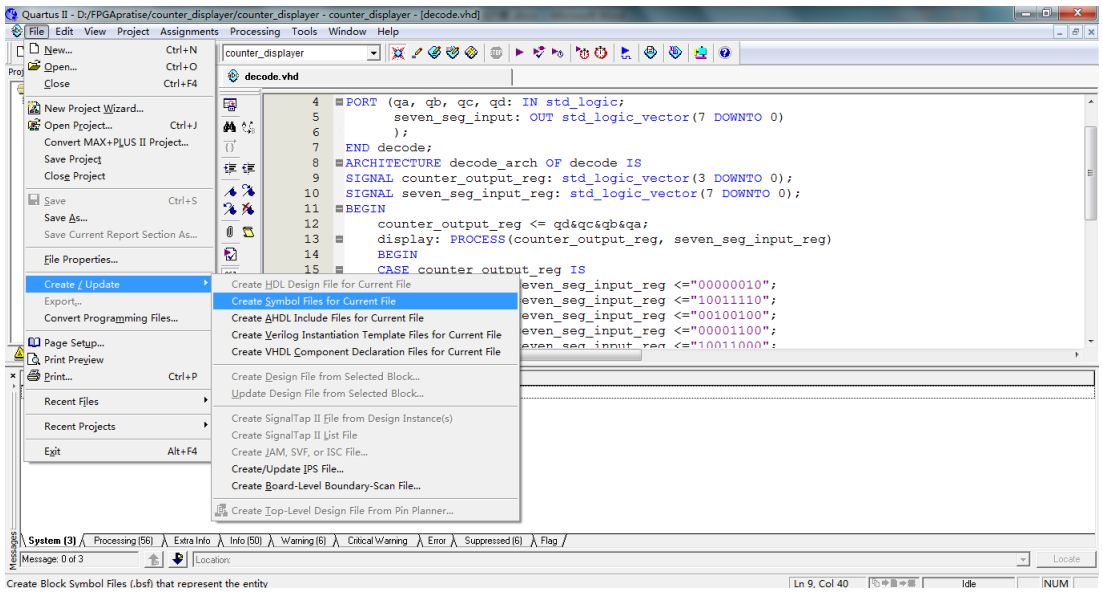


图16　生成 block

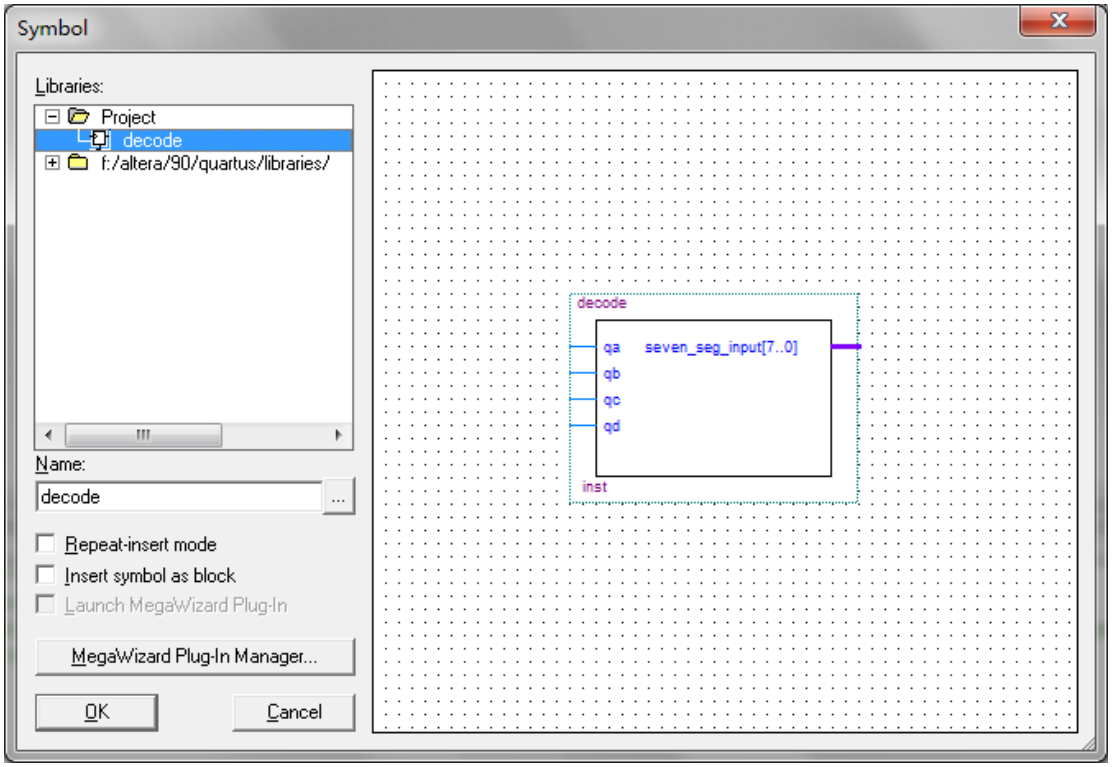


图17　自动添加到元件库的 decode 模块

此模块的设计过程可以参考译码器电路的设计过程。首先新建个.vhd 文件，保存为 counter.vhd 文件，然后在此文件中编辑代码实现功能，接着编译并为此文件创建一个 block 块，程序如下：

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
ENTITY count_output IS
PORT (a, b, c, d: IN std_logic;
      result: OUT std_logic_vector(3 DOWNTO 0)
      );
END count_output;
ARCHITECTURE count_output_arch OF count_output IS
  SIGNAL result_reg:std_logic_vector(3 DOWNTO 0);
BEGIN
    result_reg <= d&c&b&a;
```

```
    result<=result_reg;
END count_output_arch;
```

这个模块如图18 所示。

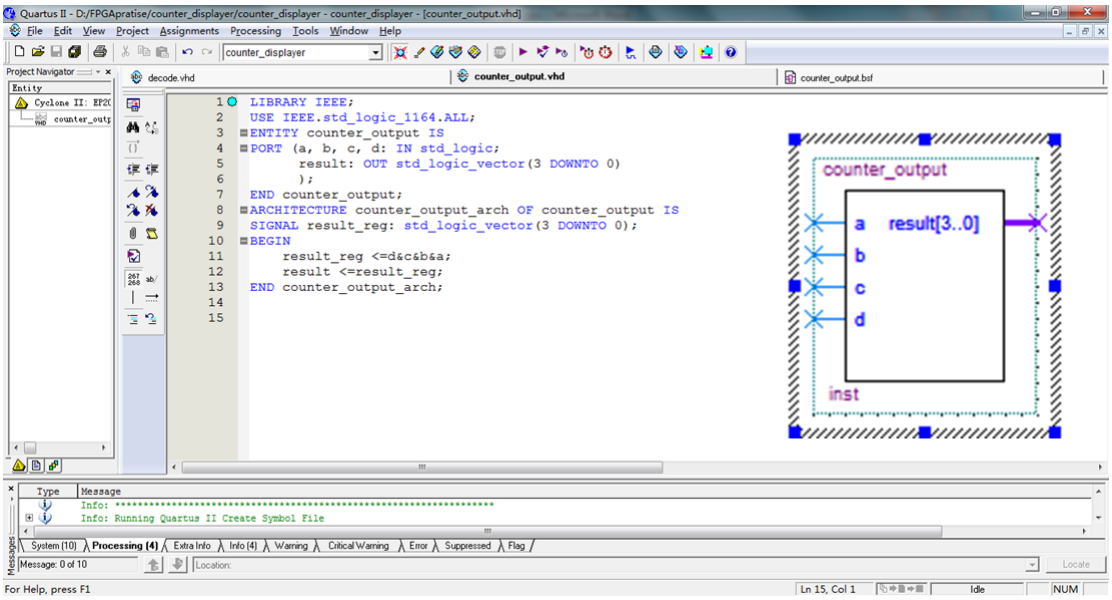


图18 counter output 模块

(3)建立顶层文件在步骤 1）和 2）中，都是先在文本编辑器中编辑程序，编译通过后再生成一个与之对应的模块，这两个模块在工程文件夹中的名字分别是：decode.bsf 和 counter_output.bsf，下面需完成此设计的最后一步，点击 File-> new 后,在弹出的对话框中(见图12） 选择 Block Diagram/Schematic File，用图形编辑器来新建一个图形输入文件。双击 Block Diagram/Schematic File 后，就进入了图形输入的界面，如图19 所示，将名为 block.bdf 的文件保存为 counter_displayer.bdf， 保存后的文件 counter_displayer.bdf 即为整个工程 的 Top-Level Entity。

在界面空白处双击左键，会弹出如图20 左边所示的器件模块选择框，在图中,我们看到先前生成的两个模 块:decode 以及 counter_output 都在 Project 下 ， 依 次 双 击 decode 、 counter_output 以 及 图 中 c ： /altera/quartus51/libraries-> others-> maxplus2-> 74161，将这三个模块 添加进图形编辑环境中，分别如

图20、图21、22 所示。

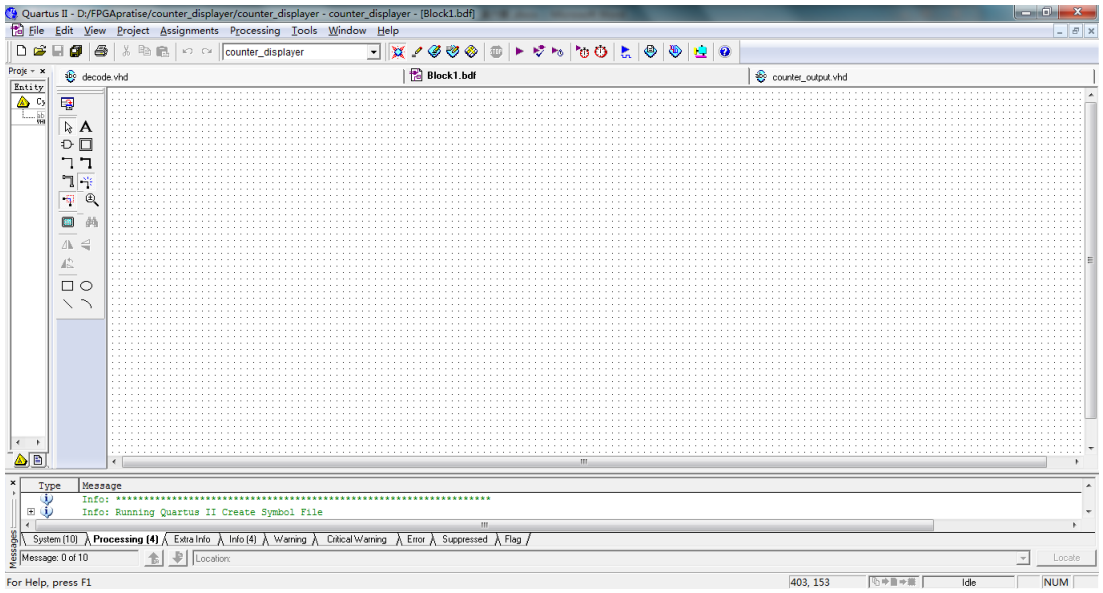


图19 图形输入界面

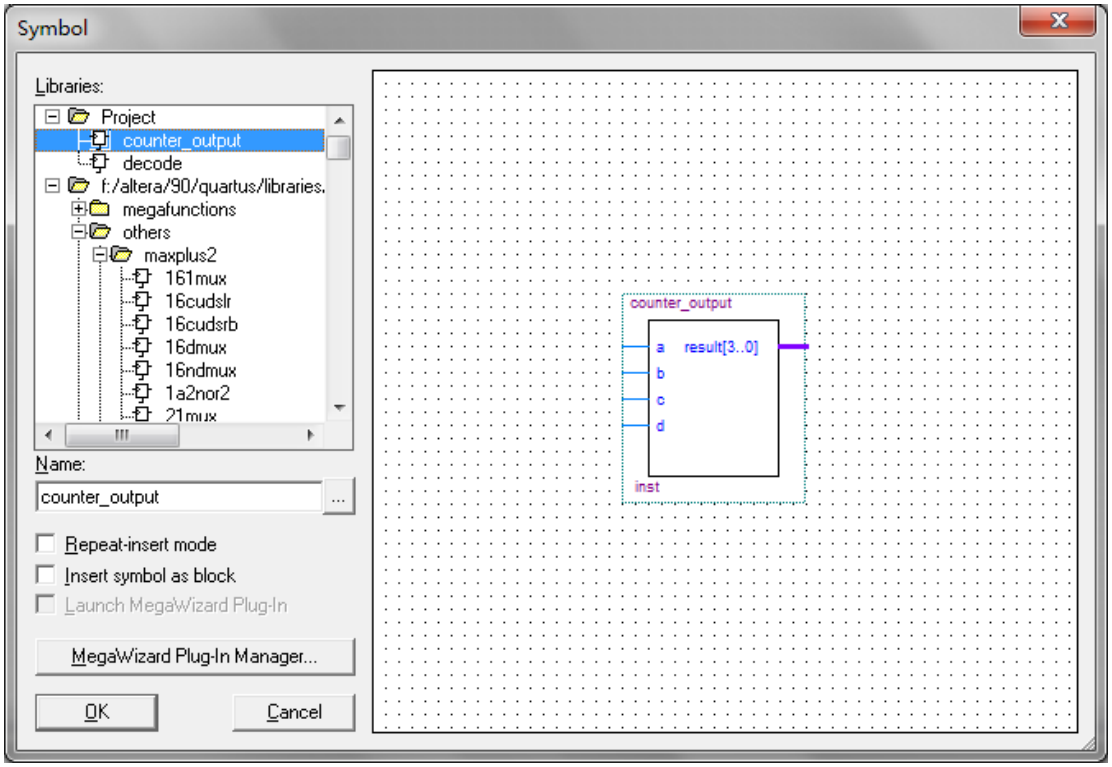


图20 选择 counter_output 模块

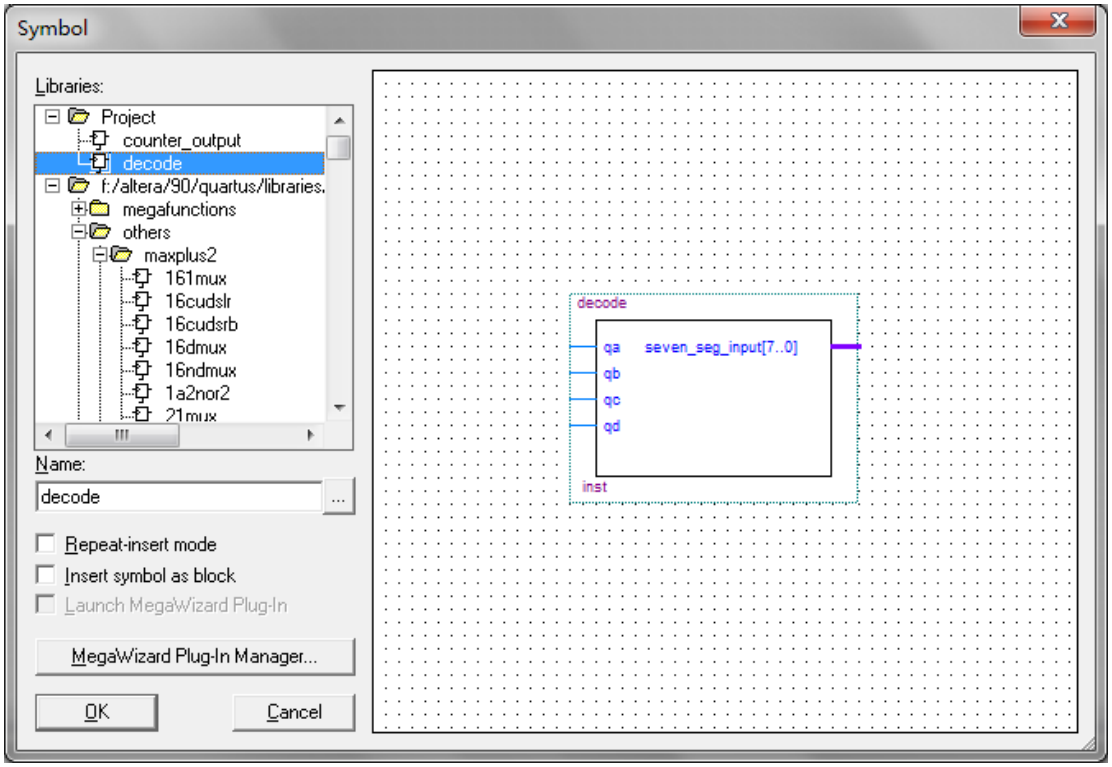


图21　选择 decode 模块

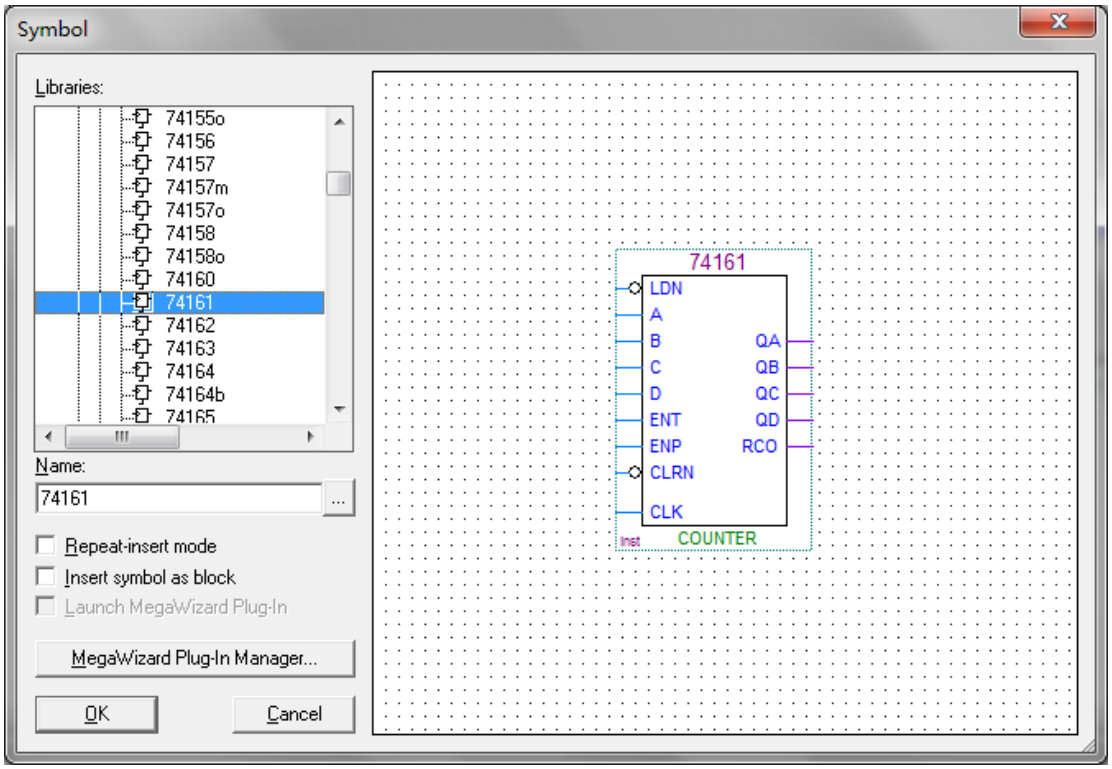
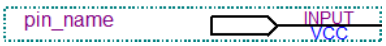
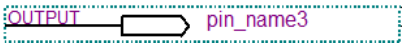


图22　选择 74161 模块

注意：各个模块的后缀名是.bsf 块符号文件，而工程的 counter_displayer 的 top-level 实体的后缀是.bdf 的设计文件。

添加完毕三个模块之后，还应添加输入输出符号，接下来的工作就是用信号线把模块之间对应的端口连接起来，且系统的输入信号与输入符号相连，系统的输出信号与输出符号相连。

输入符号： 

输出符号： 

连接好的整个系统如图23 所示。

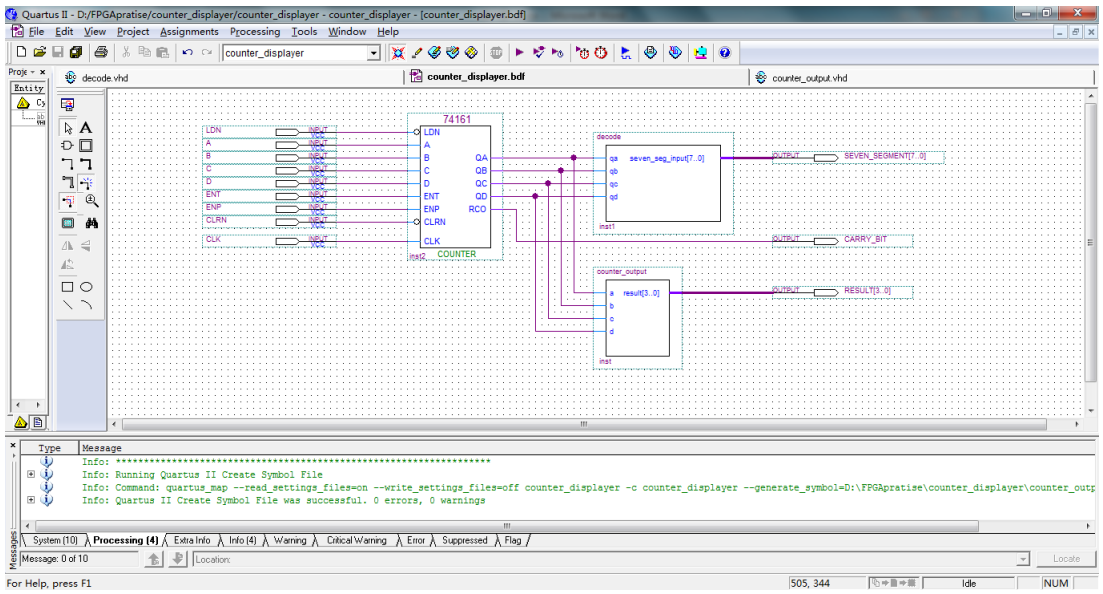


图23　连接好的整个系统

整个系统的输入信号分别是：LDIN（预置输入使能）、A、B、C、D（A、B、C、D 为预置输入端）、ENT、ENP、CIRN（置零使能）、CLK（时钟输入端），此计数器的工作方式请参见 74161 的功能表。 图23 的连线中，单根信号线由连线工具来画出，而信号总线由连线工具 来画出，双击输入输出符号可以改变输入输出 信号 的名字。要删除某根信 号线，可以先用鼠标单击选中要删除的信号 线，然后按 Delete 键就可以删除了。在对总线重命名的时候应注意要标明总线的宽度如 result[3..0]，不然在编译的时候也会出错。

3. 编译

连接好并重命名之后，保存文件，然后就可以对这个顶层文件counter_displayer.bdf进行编译了，编译完将出现如图23所示的警告。

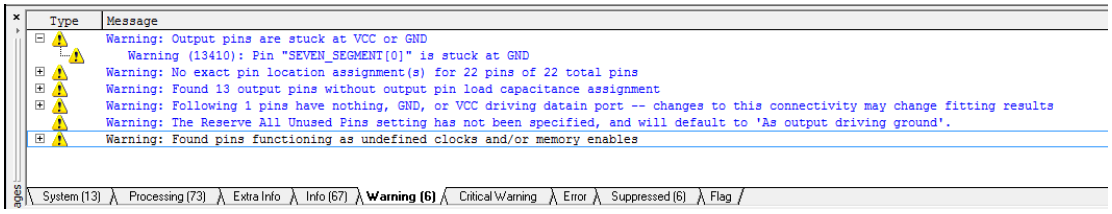


图24　编译后出现警告

警告内容：

a)Warning: Output pins are stuck at VCC or GND

Warning (13410): Pin "SEVEN_SEGMENT[0]" is stuck at GND

b)Warning: No exact pin location assignment(s) for 22 pins of 22 total pins

Warning: Found 13 output pins without output pin load capacitance assignment

Warning: Following 1 pins have nothing, GND, or VCC driving datain port -- changes to this connectivity may change fitting results

c)Warning: The Reserve All Unused Pins setting has not been specified, and will default to 'As output driving ground'.

d)Warning: Found pins functioning as undefined clocks and/or memory enables

Info: Assuming node "CLK" is an undefined clock

出现的警告没有大碍。警告 a）指出有信号始终接地，因为在译码器电路中输出信号总线 SEVEN_SEGMENT[7..0]中的 SEVEN_SEGMENT(7)始终是 0，对整个设计没有影响；警告 b）说明管脚尚未分配，后面管脚分配后这个警告就会消失；警告 c）指出未分配的管脚尚未指定状态故默认为输出接地，我们一般需要在 Setting 中设定为三态输出从而消除警告；警告 d）在配置管脚时自动解除，对仿真也不会有什么影响。

4. 时序仿真

首先建立波形文件，点击 File ->New->Other Files，在出现的对话框中双击 Vector Waveform File 使进入了波形文件编辑环境。在

图25 中，右键点击左边空白处，选择 Insert Node or Bus…，添加用以仿真的输入输出信号。需要注意的是，Quartus II 软件自 10.0 版本后就不再自带仿真功能了，相关功能可用 Modelsim 实现， 这里仅以 Quartus II 9.0 版本为例进行讲解。

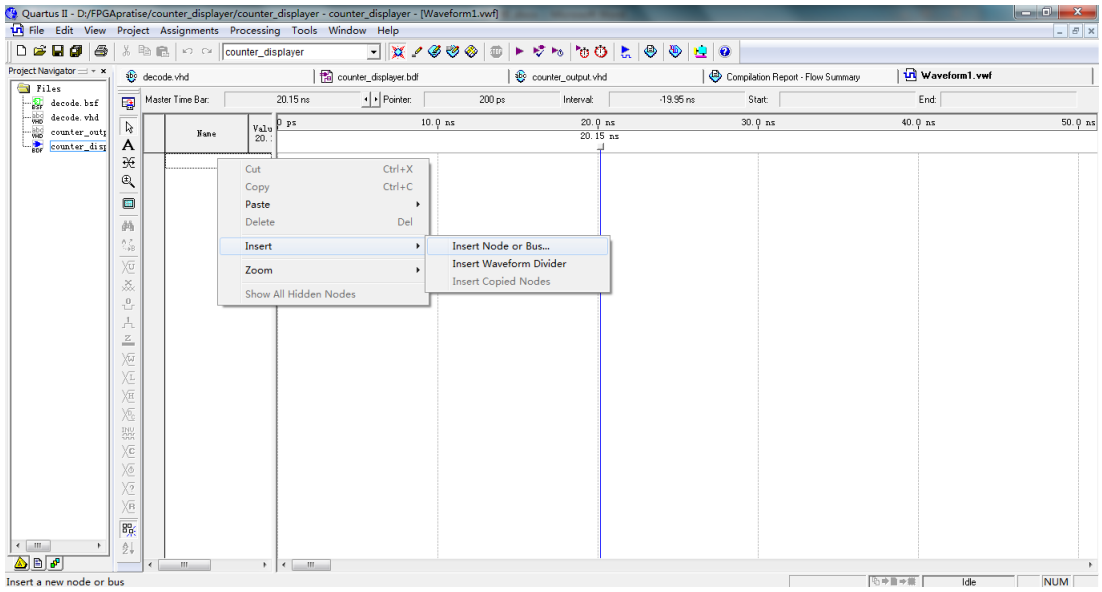


图25 波形文件编辑环境

在出现的 Insert Node or Bus 对话框中点击 Node Finder 后，即会出现一个为仿真添加输入输出信号的界面，在界面的 Filter 中选择 Post-Complication 在点击 List，那么工程编译后所有内外部的信号都会被列出来，这里所指的外部信号即系统的所有输入输出信号，而内部信号则是系统内的一些寄存器的输入输出信号，不同进程间用于通信的信号等，一般只选择输入输出信号即可，如果需要更详细的分析系统，则通常也将与寄存器相关的一些信号添加到仿真波形文件中。

系统的所有信号如图26 左边所示。

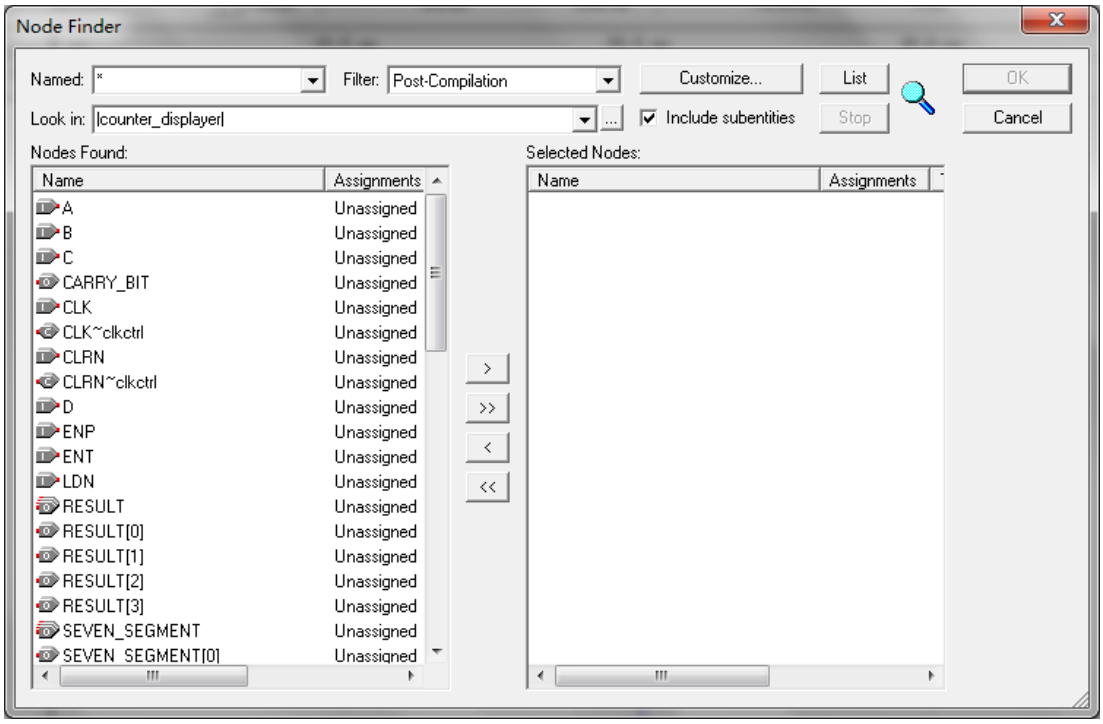


图26　添加输入输出信号界面

选择需添加到波形文件的信号后点击 ok，工程的波形文件如图27 所示。

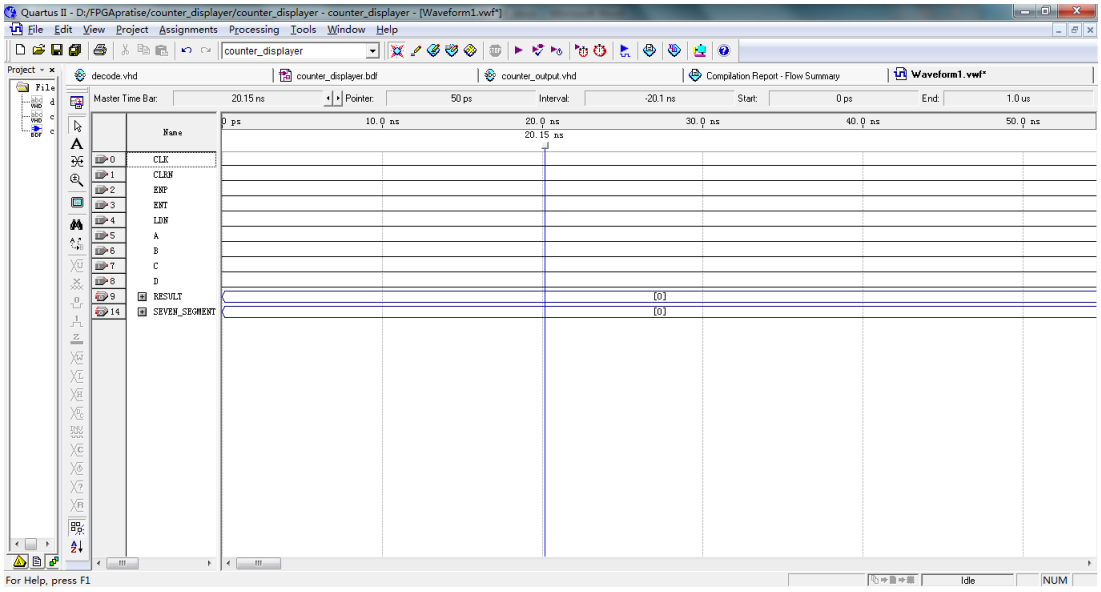


图27　工程的波形文件

下一步工作是为波形文件中每个输入信号编辑激励信号，编辑激励信号的工具如图26 的最左端所示，如欲编辑输入时钟信号 clk，则只需要先点击 clk 信号，使之变为浅蓝色，然后运用左端的编辑工具，输入它的周期，即时钟的频率；而对于其他输入信号一般用：、就可以完成对其编辑。

编辑好输入波形文件，保存后便可用于仿真。仿真结果如

图28 所示，经观察，仿真结果符合设 计要求。

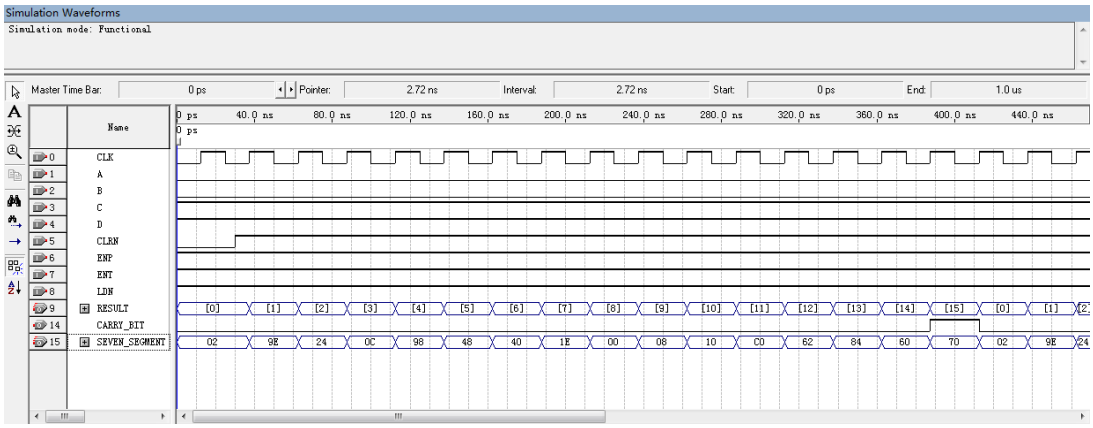


图28　仿真结果

实验二 基本电路模块设计（组合、时序）

一 实验目的：

用 VHDL 语言设计组合逻辑、时序逻辑电路模块

二 实验内容

前期准备：

1) 接上 usb-blaster 到开发板中间的 JTAG 接口，另一个接口接上电脑 USB，然后接上电源，为开发板通电。如果驱动安装成功则进行下个步骤，如果驱动没有安装成功则按照‘USB 下载线驱动安装指南.xdoc’为 usb-blaster 安装驱动。驱动安装成功则开始按照实验要求进行系统设计。
2) 本次实验所使用开发板芯片信号为 cyclone Ⅱ系列的 EP2C8Q208C8
3) 下载的步骤见附录。

1、3-8 译码器

设计要求：

1) 3 个译码输入端 A、B、C ，分别对应开发板上表示为 ON DIP 的 2,3,4
2) EN 为使能端（低电平有效），对应开发板上表示为 ON DIP 的 1 管脚
3) Y 为译码输出，8 位位矢量类型。D3 至 D10
4) 输入采用电平开关，译码输出采用 LED 指示灯显示

5）管脚对应见附件：信号分配表

6）译码表如下：

A	B	C	EN	Y							
X	X	X	1	1	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	1	1	1	0
0	0	1	0	1	1	1	1	1	1	0	1
0	1	0	0	1	1	1	1	1	0	1	1
0	1	1	0	1	1	1	1	0	1	1	1
1	0	0	0	1	1	1	0	1	1	1	1
1	0	1	0	1	1	0	1	1	1	1	1
1	1	0	0	1	0	1	1	1	1	1	1
1	1	1	0	0	1	1	1	1	1	1	1

2、分频器实验

设计要求：

1) 将 EDA 板上的系统时钟 50MHz 分频为 1Hz 的时钟信号
2) 占空比为 50%
3) 利用流水灯点亮程序，在 EDA 板上观察效果
4) 实体命名为 clkdiv_(班级号)_(班级序号)

注：由于开发板上没有 rst_n 按钮，同学们将 rst_n 直接接高电平。

注：下面为一个流水灯点亮程序实例，供参考

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity led is
  port(led_out : out std_logic_vector(7 downto 0);
           clk : in std_logic;
         rst_n : in std_logic
        );
end led;

architecture behavior of led is
  signal light : std_logic_vector (7 downto 0);
 begin
  process(clk,rst_n)
     begin
       if(rst_n = '0')then
            light <= "00000010";
       elsif(clk'event and clk ='1' )then
        if(light = "00000000" ) then
            light <= "00000001";
        else
            if(light = "10000000" ) then
                 light <= "00000001";
            else
                 light <= light(6 downto 0)& '0';
            end if;
        end if;
       end if;
   end process;
   led_out <= light;

end behavior;
```

附：实验二所需资料

特别注意事项： 1.由于实验室第一次新装 WINDOWS7 版本问题,后面每次数字系统设计实验时,开启电脑时,请在进入屏幕刚出现文字时,按 F8 键后(不能过早也不能太迟按),用鼠标将光标移到最下一行:"禁止驱动软件数字签名"栏,回车,即可。2.第二次实验课包括两个单独的实验，请一定要提前预习，写好源程序，在宿舍做好时序仿真,否则可能上课完成不了任务。上课时给开发板实际演示。开发板所有资料都在“新板”附件中，其中管脚配置以“新板”附件中为准，SW0、1、2、3 为按键；LED0-5 为 LED 灯。LED6，7 灯板上没有，可空配置为 193

197 脚。

一，USB-Blaster 驱动安装方法

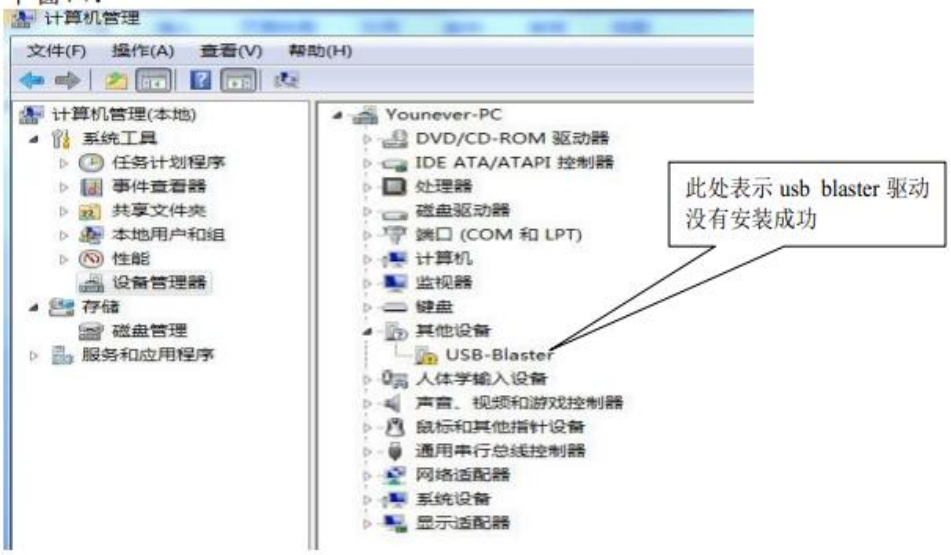
在安装完Quartus II软件后会在QII软件安装目录的:
\altera\quartus10.0\drivers\usb-blaster找到USB下载线的驱动

驱动安装步骤:

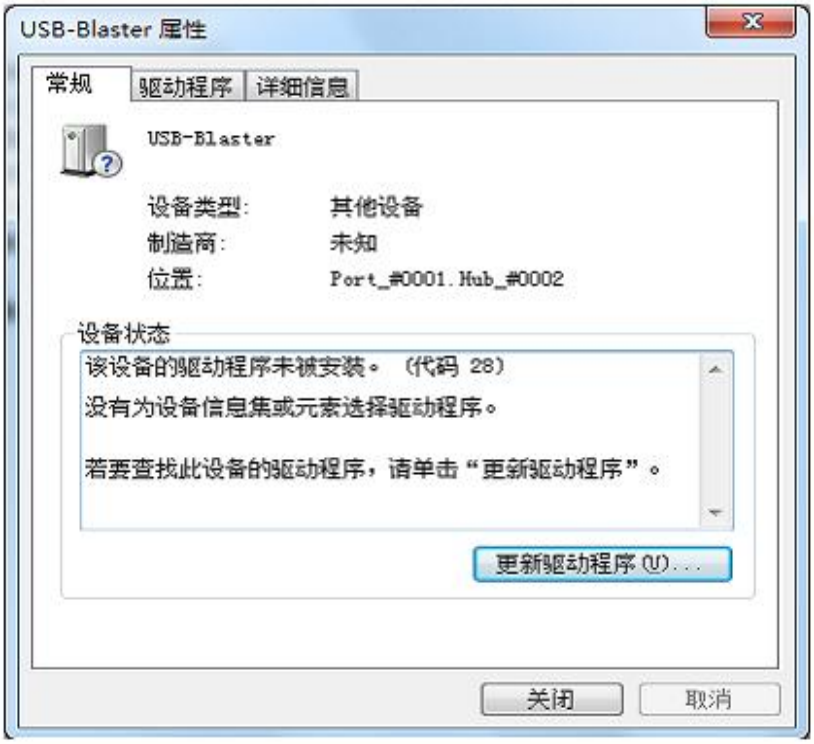
1、首先插上USB下载线,EDA开发板上电，如果电脑自带驱动能够驱动成功则跳过该步骤，没有提示驱动安装则进行第二步操作。

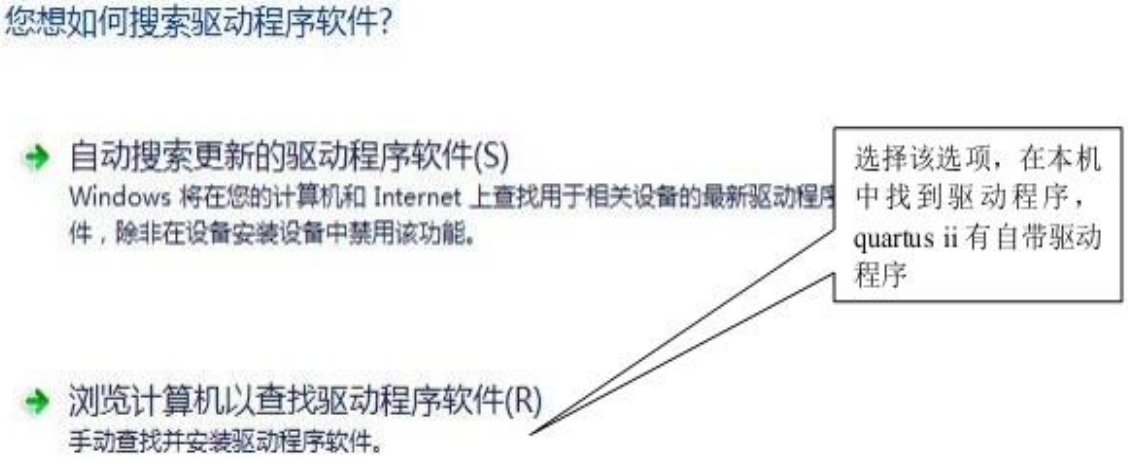
2、如果不能自动安装驱动则按照以下步骤进行驱动的安装：

1） 右击‘我的电脑’找到‘管理’，左键单击设备管理器，出现以下窗口：

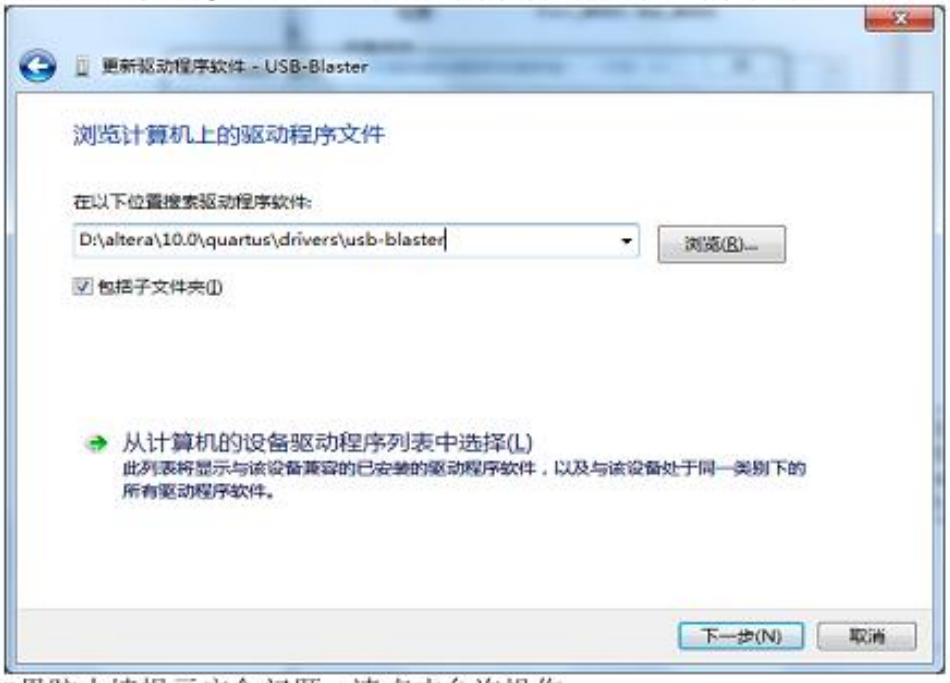


2） 双击usb-blaster出现以下窗口，单击更新驱动程序



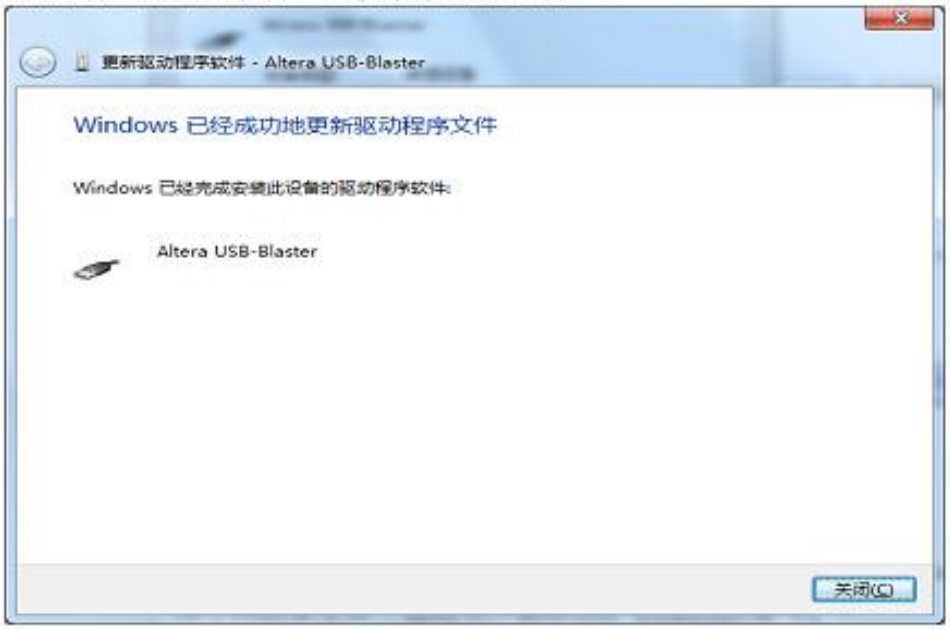


3） 在电脑硬盘安装quartus ii的目录中找到驱动程序。路径如下：



4） 如果防火墙提示安全问题，请点击允许操作。

5） 安装成功，出现以下窗口，接下来重启电脑：



实验三 基于状态机的交通灯控制

一 实验目的：运用状态机设计红黄绿交通灯控制电路系统

二 实验内容：

1、 开发板上三个 led 等分别代表公路上红黄绿三种颜色交通灯。
2、 交通灯状态机初始状态为红灯，交通灯工作过程依次是红 ⇨绿⇨黄 ⇨ 红。
3、 为了方便观察，本次实验要求红灯的显示时间为 9s，绿灯显示时间为 6s ，黄灯显示时间为 3s ，时间需要倒计时，在数码管上显示。编程之前要求同学们先画好 ASM 图。
4、 1Hz 分频模块请采用第二次实验中的内容，7 段码显示模块请参考书本相关内容。
5、 第三次实验课用到 EP2C8Q208C8 通过 74HC595 驱动数码管，有两种方法写该模块代码：方法 1，用 VHDL 语言写，（自己写 VHDL 代码有加实验分）。方法 2，可调用 verilog 数码管驱动模块，该模块在附件“seg.zip”中。和其它 VHDL 编写的模块可以混搭在一个电路图中使用。EP2C8Q208C8 的 SCTP，SHCP，SER__DATA 数码管信号线通过两块 74HC595 集成块，再驱动数码管。
6、 芯片型号：cyclone:EP2C8Q208C8,开发板所有资料都在“新板”附件中，其中管脚配置在实验要求中是不对的，以“新板”附件中为准。

附：八段 led 译码列表

	A	B	C	D	E	F	G	H
0	0	0	0	0	0	0	1	0
1	1	0	0	1	1	1	1	0
2	0	0	1	0	0	1	0	0
3	0	0	0	0	1	1	0	0
4	1	0	0	1	1	0	0	0
5	0	1	0	0	1	0	0	0
6	0	1	0	0	0	0	0	0
7	0	0	0	1	1	1	1	0
8	0	0	0	0	0	0	0	0
9	0	0	0	0	1	0	0	0

附：供参考的 verilog 数码管驱动代码

```
module seg(
          clk,rst_n,en,
          idis_data,
          ds_stcp,ds_shcp,ds_data
       );

input clk;//25M??????
input rst_n;  //??????????
input en;
input [15:0] idis_data ;

output ds_stcp;      //74HC595??????????????????????????
output ds_shcp;      //74HC595??????????????????????
output ds_data;      //74HC595???????



//????? 0~F ??????
parameter SEG_NUM0   = 8'hc0,
          SEG_NUM1   = 8'hf9,
          SEG_NUM2   = 8'ha4,
          SEG_NUM3   = 8'hb0,
          SEG_NUM4   = 8'h99,
          SEG_NUM5   = 8'h92,
          SEG_NUM6   = 8'h82,
          SEG_NUM7   = 8'hF8,
          SEG_NUM8   = 8'h80,
          SEG_NUM9   = 8'h90,
          SEG_NUMA   = 8'h88,
          SEG_NUMB   = 8'h83,
          SEG_NUMC   = 8'hc6,
          SEG_NUMD   = 8'ha1,
          SEG_NUME   = 8'h86,
          SEG_NUMF   = 8'h8e;

//????? 0~7????
parameter SEG_WE0       = 8'b1111_1110,
          SEG_WE1       = 8'b1111_1101,
          SEG_WE2       = 8'b1111_1011,
          SEG_WE3       = 8'b1111_0111;
       //  SEG_WE4       = 8'b1110_1111,
       //  SEG_WE5       = 8'b1101_1111,
```

```
        //  SEG_WE6       = 8'b1011_1111,
        //  SEG_WE7       = 8'b0111_1111;

wire en;
reg clk_div_2;
reg clk1;

always@(en)  //use enable siganl to break the module
begin
    if (en==1)
    clk1<=clk;
    else clk1<=clk1;
end


always@(posedge clk1 or negedge rst_n)
    if(!rst_n)
        clk_div_2<=1'b0;
    else
        clk_div_2<=~clk_div_2;

//-------------------------------------------------



//-------------------------------------------------
//??????????
reg[3:0] seg_num;//??????
reg[7:0] seg_duan;   //7??????????????8??
reg[7:0] seg_wei;//7????????

reg[7:0] cnt_4;      //?????

    //?????
always @(posedge clk_div_2 or negedge rst_n)
    if(!rst_n) cnt_4 <= 8'd0;
    else cnt_4 <= cnt_4+1'b1;

    //????
always @(posedge clk_div_2 or negedge rst_n)
    if(!rst_n) seg_num <= 8'h00;
    else
        case(cnt_4[7:6])
               2'b00: seg_num <= idis_data[3:0];
```

```
            2'b01: seg_num <= idis_data[7:4];
            2'b10: seg_num <= idis_data[11:8];
            2'b11: seg_num <= idis_data[15:12];
        default:  ;
        endcase

   //??????
reg flag;
always @(posedge clk_div_2 or negedge rst_n)
   if(!rst_n) begin seg_duan <= 8'hff;
                //  flag<=1'b0;
                end
   //else if(flag) begin seg_duan<=8'hff;
        //        flag<=~flag;
            //  end
   else
      case(seg_num)
         4'h0: seg_duan <= SEG_NUM0;
         4'h1: seg_duan <= SEG_NUM1;
         4'h2: seg_duan <= SEG_NUM2;
         4'h3: seg_duan <= SEG_NUM3;
         4'h4: seg_duan <= SEG_NUM4;
         4'h5: seg_duan <= SEG_NUM5;
         4'h6: seg_duan <= SEG_NUM6;
         4'h7: seg_duan <= SEG_NUM7;
         4'h8: seg_duan <= SEG_NUM8;
         4'h9: seg_duan <= SEG_NUM9;
         4'ha: seg_duan <= SEG_NUMA;
         4'hb: seg_duan <= SEG_NUMB;
         4'hc: seg_duan <= SEG_NUMC;
         4'hd: seg_duan <= SEG_NUMD;
         4'he: seg_duan <= SEG_NUME;
         4'hf: seg_duan <= SEG_NUMF;
      default:    ;
      endcase


   //????
always @(cnt_4[7:6])
   case(cnt_4[7:6])
         2'b00: seg_wei <= SEG_WE0;
         2'b01: seg_wei <= SEG_WE1;
         2'b10: seg_wei <= SEG_WE2;
         2'b11: seg_wei <= SEG_WE3;
```

```
        default:  seg_wei <= 8'b0000_0000;
        endcase

//-----------------------------------------------
//74HC95????
reg ds_stcpr; //74HC595??????????????????????????
reg ds_shcpr; //74HC595??????????????????????
reg ds_datar; //74HC595???????

    //????????
always @(posedge clk_div_2 or negedge rst_n)
    if(!rst_n) ds_shcpr <= 1'b0;
    else if((cnt_4 > 8'h02 && cnt_4 <= 8'h22) || (cnt_4 > 8'h24 && cnt_4
<= 8'h44)
            || (cnt_4 > 8'h46 && cnt_4 <= 8'h66) || (cnt_4 > 8'h68 && cnt_4
<= 8'h88)
            || (cnt_4 > 8'h90 && cnt_4 <= 8'hb0) || (cnt_4 > 8'hb2 && cnt_4
<= 8'hd2)
            || (cnt_4 > 8'hd4 && cnt_4 <= 8'hf4))
        ds_shcpr <= ~ds_shcpr;
    else ds_shcpr<=1'b0;

    //????????
always @(posedge clk_div_2 or negedge rst_n)
    if(!rst_n) ds_datar <= 1'b0;
    else
        case(cnt_4)
            8'h02,8'h46,8'h90,8'hd4: ds_datar <= seg_duan[7];
            8'h04,8'h48,8'h92,8'hd6: ds_datar <= seg_duan[6];
            8'h06,8'h4a,8'h94,8'hd8: ds_datar <= seg_duan[5];
            8'h08,8'h4c,8'h96,8'hda: ds_datar <= seg_duan[4];
            8'h0a,8'h4e,8'h98,8'hdc: ds_datar <= seg_duan[3];
            8'h0c,8'h50,8'h9a,8'hde: ds_datar <= seg_duan[2];
            8'h0e,8'h52,8'h9c,8'he0: ds_datar <= seg_duan[1];
            8'h10,8'h54,8'h9e,8'he2: ds_datar <= seg_duan[0];
            8'h12,8'h56,8'ha0,8'he4: ds_datar <= seg_wei[0];
            8'h14,8'h58,8'ha2,8'he6: ds_datar <= seg_wei[1];
            8'h16,8'h5a,8'ha4,8'he8: ds_datar <= seg_wei[2];
            8'h18,8'h5c,8'ha6,8'hea: ds_datar <= seg_wei[3];
            8'h1a,8'h5e,8'ha8,8'hec: ds_datar <= seg_wei[4];
            8'h1c,8'h60,8'haa,8'hee: ds_datar <= seg_wei[5];
            8'h1e,8'h62,8'hac,8'hf0: ds_datar <= seg_wei[6];
            8'h20,8'h64,8'hae,8'hf2: ds_datar <= seg_wei[7];
```

```
            8'h24,8'h68,8'hb2,: ds_datar <= 1;
            8'h26,8'h6a,8'hb4,: ds_datar <= 1;
            8'h28,8'h6c,8'hb6,: ds_datar <= 1;
            8'h2a,8'h6e,8'hb8,: ds_datar <= 1;
            8'h2c,8'h70,8'hba,: ds_datar <= 1;
            8'h2e,8'h72,8'hbc,: ds_datar <= 1;
            8'h30,8'h74,8'hbe,: ds_datar <= 1;
            8'h32,8'h76,8'hc0,: ds_datar <= 1;
            8'h34,8'h78,8'hc2,: ds_datar <= 1;
            8'h36,8'h7a,8'hc4,: ds_datar <= 1;
            8'h38,8'h7c,8'hc6,: ds_datar <= 1;
            8'h3a,8'h7e,8'hc8,: ds_datar <=1;
            8'h3c,8'h80,8'hca,: ds_datar <= 1;
            8'h3e,8'h82,8'hcc,: ds_datar <= 1;
            8'h40,8'h84,8'hce,: ds_datar <= 1;
            8'h42,8'h86,8'hd0,: ds_datar <= 1;

            default: ds_datar <= seg_duan[0];
            endcase

    //????????
always @(posedge clk1 or negedge rst_n)
    if(!rst_n) ds_stcpr <= 1'b0;
    else if((cnt_4 == 8'h02) || (cnt_4 == 8'h23) || (cnt_4 == 8'h45) ||
(cnt_4 == 8'h67) || (cnt_4 == 8'h89)|| (cnt_4 == 8'hb1)|| (cnt_4 == 8'hd3))
ds_stcpr <= 1'b0;
    else if((cnt_4 == 8'h22) || (cnt_4 == 8'h44) || (cnt_4 == 8'h66) ||
(cnt_4 == 8'h88) || (cnt_4 == 8'hb0)|| (cnt_4 == 8'hd2)|| (cnt_4 == 8'hf4))
ds_stcpr <= 1'b1;

wire ds_stcp = ds_stcpr;
wire ds_shcp = ds_shcpr;
wire ds_data = ds_datar;

endmodule
```

实验四 按键控制的状态机设计

一 **实验目的**：运用状态机设计按键控制数码管显示的电路系统

二 **实验内容**：

设计要求：

1、 按键控制数码管显示，当按下S2时，Q4数码管显示1；再次按下S2，Q4数码管显示2；第三次按下S2，Q4数码管显示3；第四次按下S2时，Q4数码管显示4.第五次按下S2时Q4又从1开始显示，如此反复。

2、 检测按键是否按下需要消抖，消抖程序请同学们参考附录程序，参考程序是利用四个按键控制四个LED灯的亮灭。请同学们在读懂参考程序的基础上将其改编为符合要求1的程序。

4、 参考程序是普通的控制程序，本实验要求同学们利用状态机来编写程序。

附：八段 led 数码管编码表（1~4）

	A	B	C	D	E	F	G	H
1	1	0	0	1	1	1	1	0
2	0	0	1	0	0	1	0	0
3	0	0	0	0	1	1	0	0
4	1	0	0	1	1	0	0	0

参考程序

```
LIBRARY IEEE ;
USE IEEE.std_logic_1164.all ;
USE IEEE.std_logic_unsigned.all ;
ENTITY key_read IS
PORT (clk : IN std_logic ;
        sw1 ,sw2 ,sw3, sw4 :IN std_logic ;
        sw1_led ,sw2_led,sw3_led,sw4_led :BUFFER std_logic ) ;
END ENTITY ;
ARCHITECTURE ked_dec OF key_read IS
signal key_rst : std_logic_vector (3 downto 0) ;
signal key_rst_an : std_logic_vector (3 downto 0 ) ;
signal key_rst_r :std_logic_vector (3 downto 0 ) ;
signal low_sw : std_logic_vector (3 downto 0) ;
signal low_sw_r :std_logic_vector (3 downto 0 ) ;
signal low_sw_an : std_logic_vector (3 downto 0 ) ;
signal cnt : std_logic_vector (19 downto 0) ;
BEGIN
PROCESS (clk)
BEGIN
    IF clk'event and clk = '1' THEN
        key_rst <= sw4&sw3&sw2&sw1 ;
```

```
        END IF ;
END PROCESS ;
PROCESS (clk)
BEGIN
        IF clk'event and clk = '1' THEN
                key_rst_r <= key_rst ;
        END IF ;
END PROCESS ;
key_rst_an <= key_rst_r AND NOT key_rst ;
PROCESS (clk )
BEGIN
        IF clk'event and clk = '1' THEN
                IF    key_rst_an = "0000"    THEN
                        cnt <= cnt +    '1' ;
                ELSE cnt <= (OTHERS => '0') ;
                END IF ;
        END IF ;
END PROCESS ;
PROCESS (clk)
BEGIN
        IF clk'event and clk = '1'    THEN
                IF cnt = "11111111111111111111"  THEN
                        low_sw <= sw4&sw3&sw2&sw1 ;
                ELSE NULL ;
                END IF ;
        END IF ;
END PROCESS ;
PROCESS (clk )
BEGIN
        IF clk'event and clk = '1' THEN
                low_sw_r <= low_sw ;
        END IF ;
END PROCESS ;
low_sw_an <= low_sw_r AND NOT low_sw ;
PROCESS (clk)
BEGIN
        IF low_sw_an(3)='1' THEN sw4_led <= NOT sw4_led ;
        ELSIF low_sw_an(2)='1' THEN sw3_led <= NOT sw3_led ;
        ELSIF low_sw_an(1)='1' THEN sw2_led <= NOT sw2_led ;
        ELSIF low_sw_an(0)='1' THEN sw1_led <= NOT sw1_led ;
        ELSE NULL ;
        END IF ;
END PROCESS ;
END ked_dec;
```